

Program using different CUDA Device memory types and synchronization

Name: Satyam Singh

Reg no: 230905256

CSE B2 - 37

Sample Question :

Write a program in CUDA to perform tiled matrix multiplication using 2D Grid and 2D Block

Tiled matrix multiplication :

An optimization technique that breaks large matrices into smaller sub-matrices, or "tiles," which fit into fast cache or shared memory.

Reduces costly global memory accesses by reusing data, enabling significant speedups on CPUs and especially GPUs

Ans:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <cuda_runtime.h>
```

```
// Grid = (4/2, 4/2) = 2 × 2 blocks => Each block handles a 2×2  
portion of output
```

```
#define BLOCK_WIDTH 2
```

```
#define TILE_WIDTH 2
```

```
#define WIDTH 4
```

```
__global__ void MatMulElementThreadShared(int *a, int *b, int  
*c) {
```

```
__shared__ int MDs[TILE_WIDTH][TILE_WIDTH]; //Mat A 's tile
```

```
__shared__ int NDs[TILE_WIDTH][TILE_WIDTH]; //Mat B 's tile
```

```
int bx = blockIdx.x;
```

```
int by = blockIdx.y;
```

```
int tx = threadIdx.x;
```

```
int ty = threadIdx.y;
```

```

int Row = by*TILE_WIDTH + ty;
int Col= bx*TILE_WIDTH + tx;
int m, Pvalue=0;
for(m=0; m<WIDTH/TILE_WIDTH; m++) {

MDs[ty][tx] = a[Row*WIDTH + m*TILE_WIDTH + tx];
NDs[ty][tx] = b[(m*TILE_WIDTH + ty)*WIDTH+Col];
__syncthreads();
for (int k = 0; k < TILE_WIDTH; k++)
Pvalue += MDs[ty][k]*NDs[k][tx];
__syncthreads();
}
c[Row*WIDTH+Col] = Pvalue;
}

```

```

int main() {
int *matA, *matB, *matProd;
int *da, *db, *dc;

```

```
printf("\n== Enter elements of Matrix A (4x4) ==\n");
matA = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
for(int i = 0; i < WIDTH * WIDTH; i++)
scanf("%d", &matA[i]);
printf("\n== Enter elements of Matrix B (4x4) ==\n");
matB = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
for(int i = 0; i < WIDTH * WIDTH; i++)
scanf("%d", &matB[i]);
matProd = (int*)malloc(sizeof(int) * WIDTH * WIDTH);
cudaMalloc((void **) &da, sizeof(int) * WIDTH * WIDTH);
cudaMalloc((void **) &db, sizeof(int) * WIDTH * WIDTH);
cudaMalloc((void **) &dc, sizeof(int) * WIDTH * WIDTH);
cudaMemcpy(da, matA, sizeof(int) * WIDTH * WIDTH,
cudaMemcpyHostToDevice);
cudaMemcpy(db, matB, sizeof(int) * WIDTH * WIDTH,
cudaMemcpyHostToDevice);
int NumBlocks = WIDTH / BLOCK_WIDTH;
dim3 grid_conf (NumBlocks, NumBlocks);
dim3 block_conf (BLOCK_WIDTH, BLOCK_WIDTH);
```

```

MatMulElementThreadShared<<<grid_conf, block_conf>>>(da,
db, dc);

cudaMemcpy(matProd,dc,sizeof(int)* WIDTH
*WIDTH,cudaMemcpyDeviceToHost);

printf("\n-=Result of Addition=-\n");

printf("-----\n");

for (int i = 0; i < WIDTH; i++) {
for (int j = 0; j < WIDTH; j++)
printf("%6d ", matProd[i * WIDTH + j]);
printf("\n");
}

cudaFree(da);
cudaFree(db);
cudaFree(dc);

free(matA);
free(matB);
free(matProd);

return 0;
}

```

Output:

```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ nvcc sample.cu -o sample
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ ./sample

== Enter elements of Matrix A (4x4) ==
1 2 3 4
2 3 4 5
5 6 7 8
1 9 5 2

== Enter elements of Matrix B (4x4) ==
9 6 3 8
5 7 3 0
1 3 5 8
2 5 9 9

-=Result of Addition=-
-----
    30    49    60    68
    47    70    80    93
    98   133   140   168
    63    94    73    66
```

Lab Exercises:

Q1)

Write a program in CUDA to perform matrix multiplication using 2D grid and 2D block.

Ans)

```
#include <stdio.h>

#include <stdlib.h>

#include <cuda_runtime.h>

#include <device_launch_parameters.h>

#include <unistd.h>


#define BLOCK_WIDTH 2

#define TILE_WIDTH 2

#define WIDTH 4


__global__ void MatMulElementThreadShared(int *a,int *b,int *c){
__shared__ int MDs[TILE_WIDTH][TILE_WIDTH];
__shared__ int NDs[TILE_WIDTH][TILE_WIDTH];

int m;

int bx = blockIdx.x;

int by = blockIdx.y;

int tx = threadIdx.x;
```

```

int ty = threadIdx.y;

int row = by*TILE_WIDTH + ty;

int col = bx*TILE_WIDTH + tx;

int pVal = 0;

for(m=0;m<WIDTH/TILE_WIDTH;m++) {

MDs[ty][tx] = a[row*WIDTH + m*TILE_WIDTH + tx];

NDs[ty][tx] = b[(m*TILE_WIDTH + ty)*WIDTH + col];

__syncthreads();

for(int k=0;k<TILE_WIDTH;k++) pVal += MDs[ty][k]*NDs[k][tx];

__syncthreads();

}

c[row*WIDTH+col] = pVal;

}

```

```

int main(){

int *matA,*matB,*matProd;

int *dA,*dB,*dC;


printf("\n==Enter elements of Matrix A (4x4) ==\n");

matA = (int*)malloc(sizeof(int)*WIDTH*WIDTH);

for(int i=0;i<WIDTH*WIDTH;i++) scanf("%d",&matA[i]);

```



```

printf("\n==Enter elements of Matrix B (4x4) ==\n");
matB = (int*)malloc(sizeof(int)*WIDTH*WIDTH);
for(int i=0;i<WIDTH*WIDTH;i++) scanf("%d",&matB[i]);
matProd = (int*)malloc(sizeof(int)*WIDTH*WIDTH);

cudaMalloc(&dA,sizeof(int)*WIDTH*WIDTH);
cudaMalloc(&dB,sizeof(int)*WIDTH*WIDTH);
cudaMalloc(&dC,sizeof(int)*WIDTH*WIDTH);

cudaMemcpy(dA,matA,sizeof(int)*WIDTH*WIDTH,cudaMemcpyHostToDevice);
cudaMemcpy(dB,matB,sizeof(int)*WIDTH*WIDTH,cudaMemcpyHostToDevice);

int numBlocks = WIDTH/BLOCK_WIDTH;
dim3 grid(numBlocks,numBlocks);
dim3 block(BLOCK_WIDTH,BLOCK_WIDTH);

MatMulElementThreadShared<<<grid,block>>>(dA,dB,dC);
cudaMemcpy(matProd,dC,sizeof(int)*WIDTH*WIDTH,cudaMemcpyDeviceToHost);

printf("\n==Result==\n");

```

```
for(int i=0;i<WIDTH;i++){  
    for(int j=0;j<WIDTH;j++){  
        printf("%6d ",matProd[i*WIDTH+j]);  
    }  
    printf("\n");  
}  
cudaFree(dA);  
cudaFree(dB);  
cudaFree(dC);  
free(matA);  
free(matB);  
free(matProd);  
return 0;  
}
```

Output:

```
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ nvcc l10q1.cu -o q1
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ ./q1
```

```
==Enter elements of Matrix A (4x4) ==
```

```
1 2 3 4
2 3 4 5
3 4 5 6
4 5 6 7
```

```
==Enter elements of Matrix B (4x4) ==
```

```
9 8 7 6
8 7 6 5
7 6 5 4
6 5 4 3
```

```
==Result==
```

70	60	50	40
100	86	72	58
130	112	94	76
160	138	116	94

Q2)

Write a program in CUDA to improve the performance of 1D parallel convolution using constant memory

Ans)

```
#include "cuda_runtime.h"
#include <stdio.h>
#include "device_launch_parameters.h"

__constant__ float d_N[100];
__constant__ int d_width;
__constant__ float d_M[100];
__constant__ int d_mask_width;

__global__ void convolution_1D(float *P){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float Pvalue = 0;
    int N_start_point = i - (d_mask_width/2);
    for(int j=0;j<d_mask_width;j++){
        if(N_start_point+j >=0 && N_start_point+j < d_width){
            Pvalue += d_N[N_start_point+j]*d_M[j];
        }
    }
}
```

```
}  
P[i] = Pvalue;  
}
```

```
int main(void) {  
float *N, *M, *P;  
int width,mask_width;  
printf("Enter the width: ");  
scanf("%d",&width);  
printf("Enter the mask width: ");  
scanf("%d",&mask_width);  
N = (float*)malloc(width*sizeof(float));  
M = (float*)malloc(mask_width*sizeof(float));  
P = (float*)malloc(width*sizeof(float));  
float *d_c;  
int size= sizeof(float);  
for(int i=0;i<width;i++){  
N[i] = i+1;  
}  
for(int i=0;i<mask_width;i++){  
M[i] = i+10;
```

```

}

cudaMalloc((void **)&d_c, width*size);

cudaMemcpyToSymbol(d_N,N,width*size);
cudaMemcpyToSymbol(d_M,M,mask_width*size);
cudaMemcpyToSymbol(d_width,&width,sizeof(int));
cudaMemcpyToSymbol(d_mask_width,&mask_width,sizeof(int));

convolution_1D<<<1,width>>>(d_c);

cudaMemcpy(P, d_c, width*size, cudaMemcpyDeviceToHost);
printf("Result:\n");
for(int i=0;i<width;i++){
printf("%lf ",P[i]);
}
printf("\n");
cudaFree(d_c);
return 0;
}

```

Output:

```
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ nvcc l10q2.cu -o q2
• STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ ./q2
Enter the width: 2
Enter the mask width: 2
Result:
11.000000 32.000000
```

Q3)

Write a program in CUDA to perform tiled 1D convolution operation on the input array N of size width using the mask array, M of size mask_width, to produce the resultant array P of size width

Ans)

```
#include <stdio.h>
```

```
#include <cuda_runtime.h>
```

```
#define N 16 // Size of input array (can change as needed)
```

```
__global__ void inclusiveScanKernel(int *input, int *output, int n) {
```

```
__shared__ int temp[N];
```

```
int tid = threadIdx.x;
```

```

if (tid < n)
temp[tid] = input[tid];
__syncthreads();

// Inclusive scan using Hillis-Steele algorithm
for (int offset = 1; offset < n; offset *= 2) {
int val = 0;
if (tid >= offset)
val = temp[tid - offset];
__syncthreads();
if (tid < n)
temp[tid] += val;
__syncthreads();
}

if (tid < n)
output[tid] = temp[tid];
}

int main() {
int h_input[N], h_output[N];

```



```

int *d_input, *d_output;

// Initialize input array
printf("Input array:\n");
for (int i = 0; i < N; i++) {
    h_input[i] = i + 1; // Example: 1, 2, 3, ..., N
    printf("%d ", h_input[i]);
}

cudaMalloc((void**)&d_input, N * sizeof(int));
cudaMalloc((void**)&d_output, N * sizeof(int));
cudaMemcpy(d_input, h_input, N * sizeof(int),
cudaMemcpyHostToDevice);

inclusiveScanKernel<<<1, N>>>(d_input, d_output, N);
cudaMemcpy(h_output, d_output, N * sizeof(int),
cudaMemcpyDeviceToHost);
printf("\nInclusive scan output:\n");
for (int i = 0; i < N; i++) {
    printf("%d ", h_output[i]);
}

cudaFree(d_input);

```

```
cudaFree(d_output);
```

```
return 0;
```

```
}
```

OUTPUT:

```
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ nvcc l10q3.cu -o q3
STUDENT@MIT-ICT-LAB5-03:~/Desktop/230905256/Lab10$ ./q3
Input array:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Inclusive scan output:
1 3 6 10 15 21 28 36 45 55 66 78 91 105 120 136 STUDENT@MIT-ICT-LAB5-03:~/D
```